

Universitatea „Sapientia” din Cluj-Napoca  
Facultatea de Științe Tehnice și Umaniste din Târgu-Mureș  
Specializarea Calculatoare

# **Bus4U - UI**

## **PROIECT DE DIPLOMĂ**

**Coordonator științific:**

Ș.l.dr.ing. Szántó Zoltán

**Absolvent:**

Portik Szabolcs

2024

Sapientia Erdélyi Magyar Tudományegyetem  
Marosvásárhelyi Kar  
Számítástechnika szak

# **Bus4U - UI**

## **DIPLOMADOLGOZAT**

**Témavezető:**

Dr. Szántó Zoltán

Egyetemi adjunktus

**Hallgató:**

Portik Szabolcs

2024

## Kivonat

Napjaink felgyorsult világában kevesen használják a tömegközlekedést, mert az nem elég kényelmes, nem elég gyors vagy éppen nem úgy működik, ahogy azt elvárnánk tőle. A gyakori problémák közé tartozik, hogy a buszok nem tartják a menetrendet vagy a jegyek megvásárlása nehézkes. Sok helyen a menetrend nincs kifüggesztve a megállóknál és az interneten sem elérhető. Az is gyakran megesik hogy a buszok nagyon zsúfoltak és erről az utazni vágyók csak akkor vesznek tudomást, amikor már a megállóban vannak és nem áll rendelkezésre más utazási lehetőség. A Bus4U online platform ezen problémákra kínál megoldást.

Az Bus4U egy mobilalkalmazásból, egy felhasználói weboldalból és egy adminisztrátori weboldalból tevődik össze. Főbb funkciói közé tartozik a buszjáratok keresése a kiindulópont, a célállomás és az időpont megadásával, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a buszmegállók a térképen, városok és útvonalak szerint szűrve, azért, hogy a felhasználó ki tudja választani a legoptimálisabb helyet a felszálláshoz. A live map funkcióban valós időben követhetők az autóbuszok pillanatnyi helyzetei, hogy a felhasználó előre informálódni tudjon a menetrend változásáról és az esetleges pontatlanságokról. Ezen funkciók biztosításához, illetve a jegykezeléshez, szükség van egy alkalmazásra, ami a buszokon található POS terminálon fut. Ez biztosítja a jegyek érvényesítését, illetve az autóbuszok élő információinak megosztását is a központi szerverrel.

Ezen felül a webalkalmazás biztosít egy adminisztrációs felületet is a busztársaságok számára. Itt lehetőség nyílik elsősorban a menetrendek és a megállók kezelésére, valamint az útvonalak feltöltésére. A busztársaság adminisztrátora itt tudja kezelni az alkalmazottak információit, és az autóbuszok különböző adatait is, mint az időszakos műszaki vizsga és a biztosítás. Végül pedig itt található egy statisztikai felület is, ahol a busztársaságok megtekinthetik a forgalmukat, a regisztrált felhasználók számát, az eladott jegyek számát és az ezekből generált bevételt, mindezt havi, éves és mindenkor bontásban.

Továbbfejlesztési lehetőségek között szerepelnek a bérletek, illetve a különböző kedvezmények létrehozásának lehetősége, valamint a buszok telítettségének követése valós időben. A busztársaságok szempontjából fontos lehet még a sofőrök munkaidejének követése és az automatizált könyvelés bevezetése is.

Ez a dolgozat a szoftver felhasználói felületére fókuszál, a két weboldalra és a mobilalkalmazásra, míg a backend működése egy másik dolgozat keretein belül van ismertetve.

**Kulcsszavak:** tömegközlekedés, buszjárat, digitalizálás

# Tartalomjegyzék

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>1. Célkitűzések</b>                          | <b>3</b>  |
| <b>2. Hasonló alkalmazások</b>                  | <b>4</b>  |
| <b>3. A rendszer specifikációi</b>              | <b>5</b>  |
| 3.1. Felhasználói követelmények . . . . .       | 5         |
| 3.2. Rendszerkövetelmények . . . . .            | 6         |
| 3.2.1. Funkcionális követelmények . . . . .     | 6         |
| 3.2.2. Nem funkcionális követelmények . . . . . | 7         |
| <b>4. A felhasználói felület tervezése</b>      | <b>9</b>  |
| <b>5. Gyakorlati megvalósítás</b>               | <b>11</b> |
| 5.1. Felhasznált technológiák . . . . .         | 11        |
| 5.1.1. Vue.js . . . . .                         | 11        |
| 5.1.2. Vuetify . . . . .                        | 13        |
| 5.1.3. Flutter . . . . .                        | 13        |
| 5.2. Felhasználói felület . . . . .             | 13        |
| 5.2.1. Főoldal . . . . .                        | 13        |
| 5.2.2. Jegyek . . . . .                         | 17        |
| 5.2.3. Menetrend . . . . .                      | 18        |
| 5.2.4. Megállók és élő térkép . . . . .         | 19        |
| 5.3. Adminisztrátori weboldal . . . . .         | 19        |
| 5.3.1. Kezdőoldal/Statisztikák . . . . .        | 19        |
| 5.3.2. Megállók . . . . .                       | 19        |
| 5.3.3. Útvonalak . . . . .                      | 20        |
| 5.3.4. Menetrend . . . . .                      | 21        |
| 5.3.5. Buszok és alkalmazottak . . . . .        | 21        |
| <b>6. Összefoglalás</b>                         | <b>23</b> |
| 6.1. További fejlesztési lehetőségek . . . . .  | 23        |

# 1. fejezet

## Célkitűzések

Fő célunk egy olyan rendszer kialakítása, amely egyaránt hasznos a tömegközlekedést választó embereknek, illetve a busztársaságoknak. A végfelhasználó szempontjából egy olyan weboldal, illetve mobilalkalmazás elkészítése a cél, amelyben meg lehet tekinteni a menetrendet, jegyet lehet vásárolni a járatokra, meg lehet tekinteni ezek útvonalát és megállóit, valamint élőben lehet követni a buszok jelenlegi helyzetét, hogy tudni lehessen mikor érnek a hozzánk legközelebb eső megállóba.

A busztársaságok szempontjából egy webes admin felület létrehozása a cél, ahol az adminisztrátorok teljeskörűen menedzselhetik cégük működését, egyéb szoftverek bevonása nélkül. Itt az adminisztrátor feltöltheti a cég járműveinek adatait és az útvonalaikat, megjelölheti a térképen a megállóikat, szerkesztheti a menetrendeket, és a jegyárakat is beállíthatja. Ugyanezen a platformon egy összegzést találhat a profitról, a felhasználók számáról illetve az eladott jegyekről, különböző időintervallumokra leosztva.

Nem utolsó sorban egy második, kisebb alkalmazás létrehozása is a célok közé tartozik, amely a buszon lévő androidos POS terminálon fog futni, és ez szolgáltatja a buszok valós idejű helyzetét, illetve ezen lehet beszkenneálni a jegyet a buszra való felszálláskor.

## 2. fejezet

# Hasonló alkalmazások

Kutatásunk során többek között rátaláltunk a Budapest Go-ra, talán ez a meglévő megoldás áll a legközelebb a mi ötletünkhöz. Céljaink hasonlóak: hogy a rendszer egyetlen platformon biztosítsa a tömegközlekedéshez kapcsolódó különféle szolgáltatásokat, beleértve a jegyvásárlást, útvonaltervezést és a valós idejű járatinformációk megjelenítését. Lényeges különbség viszont az, hogy a Budapest Go nem rendelkezik adminisztrációs platformmal, amely lehetővé tenné a járatok, járműflotta, sofőrök és útvonalak közvetlen menedzselését, vagy legalábbis sehol nem jelenik meg róla információ.

Emellett a Budapest Go jelenleg nem kínál olyan POS terminál alkalmazást, amelyet közvetlenül a buszokon vagy villamosokon használnának jegyek és bérletek fizetésére. Az alkalmazás NFC-alapú mobilfizetést kínál, de a fizikai POS terminál integráció nem része a rendszernek. Bár az utasok számára valós idejű járatkövetés elérhető, a Budapest Go nem kínál olyan funkciót, amely a járművezetők vagy a közlekedési vállalatok számára biztosítana egy külön alkalmazást a járművek helyzetének követésére és kezelésére.

A Budapest Go egy általános közlekedési rendszer, amely a városi közlekedés minden elemét lefedi. Nincs kifejezetten busztársaságokra szabott adminisztrációs felülete, amely például specifikus buszútvonalakat, menetrendeket vagy buszvezetői beosztásokat kezelne. Mi viszont pont erre fektetnénk a hangsúlyt és egy olyan személyre szabott rendszert hoznánk létre, amelyet bármely busztársaság tudna implementálni.

## 3. fejezet

# A rendszer specifikációi

### 3.1. Felhasználói követelmények

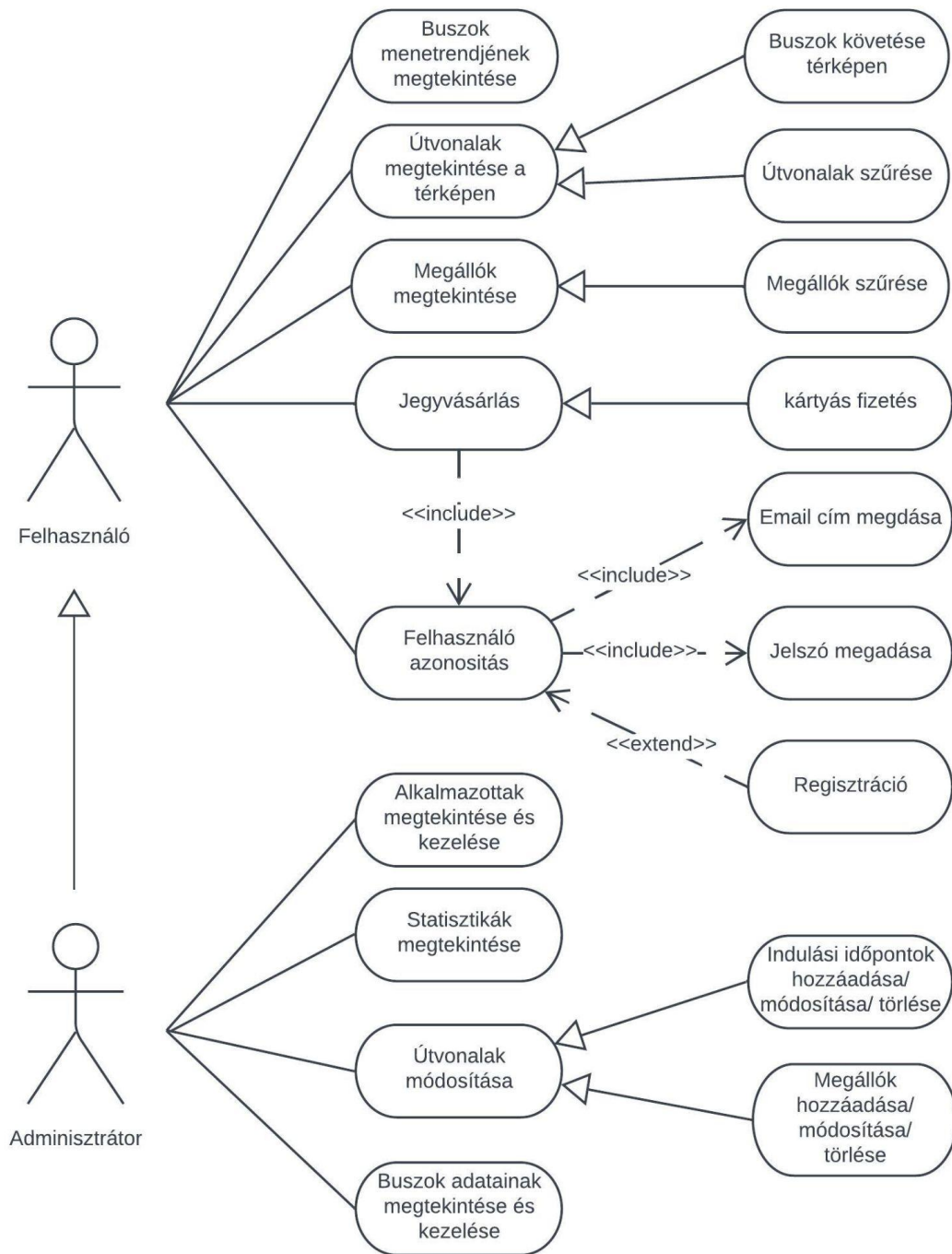
A felhasználói weboldal bejelentkezés nélkül használható. A főoldalon lehetőség nyílik rákeresni járatokra dátum, idő és cél alapján, a menetrendeknél megtekinthetők a buszok és a megállók órarendjei. A megállók oldalán megjelenik az összes megálló a térképen, amelyek között szűrést is lehet alkalmazni. Az élőben frissülő térkép lapon a követni lehet a buszok pozícióját percről percre. A jegyek oldalán vannak listázva az eddig megvásárolt jegyek QR-kódjai, a rólunk szóló oldalán pedig egy rövid leírás található a projektről. Fiókot létrehozni kizárólag a jegyvásárláshoz szükséges, ami a főoldalon lehetséges amennyiben a felhasználó megtalálta a neki megfelelő járatot és rálépett a vásárlás gombra. Fiók regisztrációjához meg kell adni a teljes nevet, egy meglévő email címet, egy új jelszót, majd egy ellenőrző kódot, amit a rendszer elküld a megadott email címre.

Az admin felület kizárólag bejelentkezéssel elérhető, új fiókokat pedig csak az adott cég főnöke tud létrehozni a saját busztársaságához, erre az alkalmazottak című lapon van lehetőség. Emellett az adminoknak lehetőségük van szerkeszteni a megállók listáját, valamint az útvonalak összes tulajdonságát is (meglátogatott megállók, indulási időpontok és jegyárak), ezeket a megállók és a járatok oldalán. A főoldalon egy statisztikát is láthatnak az eladott jegyek számáról, a megtett kilométerről és a bevételről, mindezt hónapos, éves és mindenkor bontásokban. Van még egy lap a buszok adatainak kezelésére is, ahol lehet konfigurálni az útdót, a biztosítást és a műszaki vizsgát is, valamint itt lehet buszokat kivonni forgalomból és hozzáadni újakat is.

A Flutter alapú mobil applikáció ugyanúgy működik, mint a felhasználóknak szánt weboldal, így ugyanazok a követelmények vonatkoznak erre is.

A POS terminál alkalmazás használatához szükség van egy sofőr szerepkörű admin fiókra. Bejelentkezés után lehetőség van egy adott busz követésére, illetve a jegyszkennelő programot elindítani.

A rendszer használati eset diagramja az 3.1-es ábrán látható.



3.1. ábra. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről

## 3.2. Rendszerkövetelmények

### 3.2.1. Funkcionális követelmények

A felhasználói felület minden oldalán van egy kis form, amelyet kitöltve le lehet kérni az adatbázisból az adott oldalon megjelenítendő adatokat. Ilyenre példa van a főoldalon is, ahol meg



kell adni az indulás helyét, a cél helyét (települést) és opcionálisan az ezekhez tartozó megállót, egy-egy lenyíló listából kiválasztva az adatokat. A települések betöltődnek az oldallal együtt, a megállók listája viszont csak a város kiválasztása után, mindig csak az adott településen találhatóak jelennek meg. Ezen kívül a főoldalon van egy dátumválasztó és egy időválasztó is, ezek alapértelmezetten az aktuális dátummal és idővel vannak kitöltve. Ha a form ki van töltve a keresés gomb lenyomására az oldal elküldi a bevitt adatokat a backend szerverre, amely válaszként elküld egy listát a megfelelő utazási lehetőségekkel. Ezek az adatok JSON formátumban, HTTP protokollon keresztül közlekednek a frontend és backend között. A többi oldal is hasonlóan működik, először az adatokat kell megadni, aztán a gomb lenyomására pillanatok alatt megjelenik a képernyőn a szerver válasza a bevitt adatokra.

A különböző formokon kívül, a weboldalon található térképek is API hívásokkal működnek. A rendszer először betölti az alap térképet a MapBox API segítségével, aztán amint a felhasználó kiválasztotta a megtekinteni kívánt útvonalat a Menetrend vagy az Élő térkép lapon, a weboldal a saját adatbázisunkból kéri le a hozzá tartozó megállók koordinátáit, ezeket megjeleníti a térképen, végül ismét a MapBox API-t használva útvonal-tervet generáltat. Ez úgy működik, hogy a rendszer elküldi a megállók listáját és visszakap egy sok pontból álló új listát a MapBox-tól, ezeket összekötve sok kis vonallal, kirajzolódik a részletes útvonal a térképen.

### **3.2.2. Nem funkcionális követelmények**

#### **Szervezéssel kapcsolatos követelmények**

- Programozási nyelvek: Ruby, Dart, HTML, CSS, JavaScript
- Keretrendszerek: Ruby on Rails, Flutter, Vue.js
- IDE: Visual Studio Code
- Verziókövetés: Git és GitHub
- Adatbázis: Postgres
- Cloud Service: Azure
- Webhost: Netlify
- Csoportos kommunikáció: Slack

#### **Termékkel kapcsolatos követelmények**

A felhasználói weboldal és az admin felület használatához egy modern, internetezésre alkalmas eszközre van szükség. Ajánlott a 2018 után kiadott böngészők használata:

- Chrome  $\geq 87$

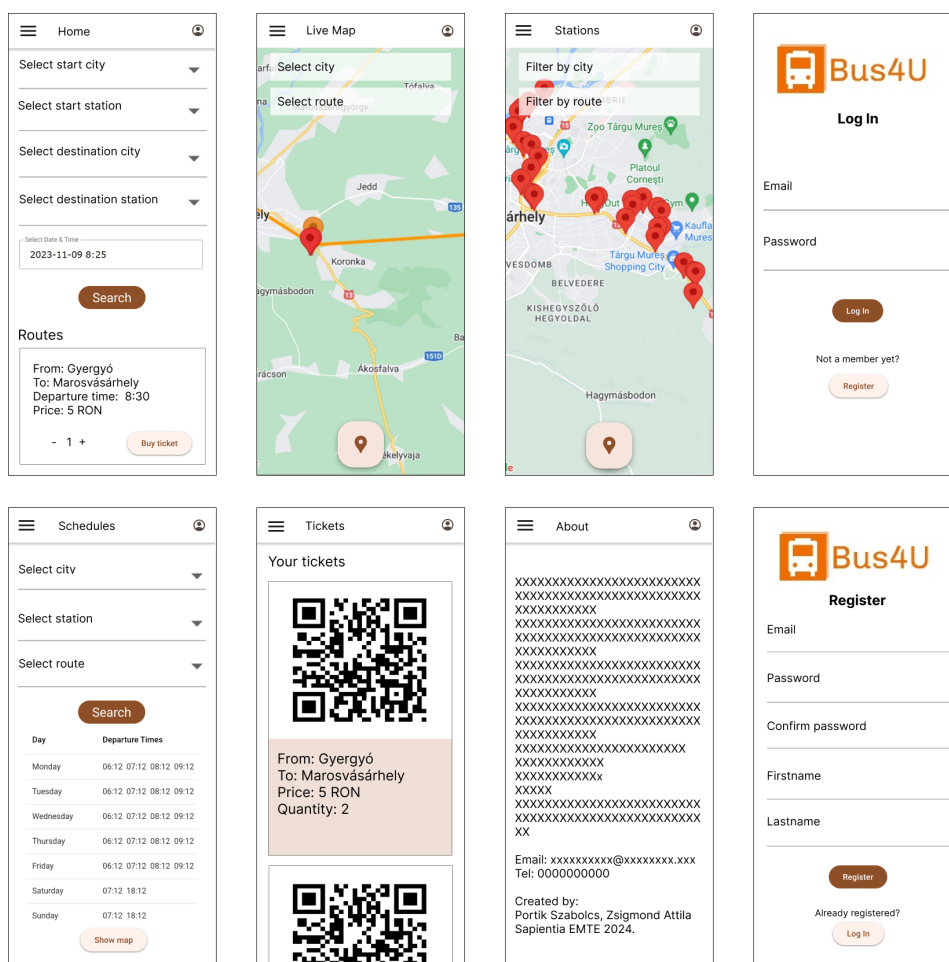
- Firefox >= 78
- Safari >= 14
- Edge >= 88

Az applikáció használatához, szükség van egy okostelefonra, vagy tabletre, amelyen minimum android 5 vagy ios 8 -as verzió van telepítve. Fontos hogy legyen rajta helymeghatározás, internet elérés (3G,4G,5G) és legyen rajta minimum 120MB szabad tárhely és 516-1024 MB szabad memória. A POS terminál alkalmazásához ideális esetben egy Sunmi V2 PRO típusú POS terminálra van szükség, de ha ez valamilyen oknál fogva nem áll rendelkezésre, akkor bármilyen, legalább Android 5.0-át, vagy iOS 8-at futtató, kamerával és GPS antennával rendelkező eszköz megteszi, illetve szükség van 100MB szabad tárhelyre és 510MB szabad memóriára.

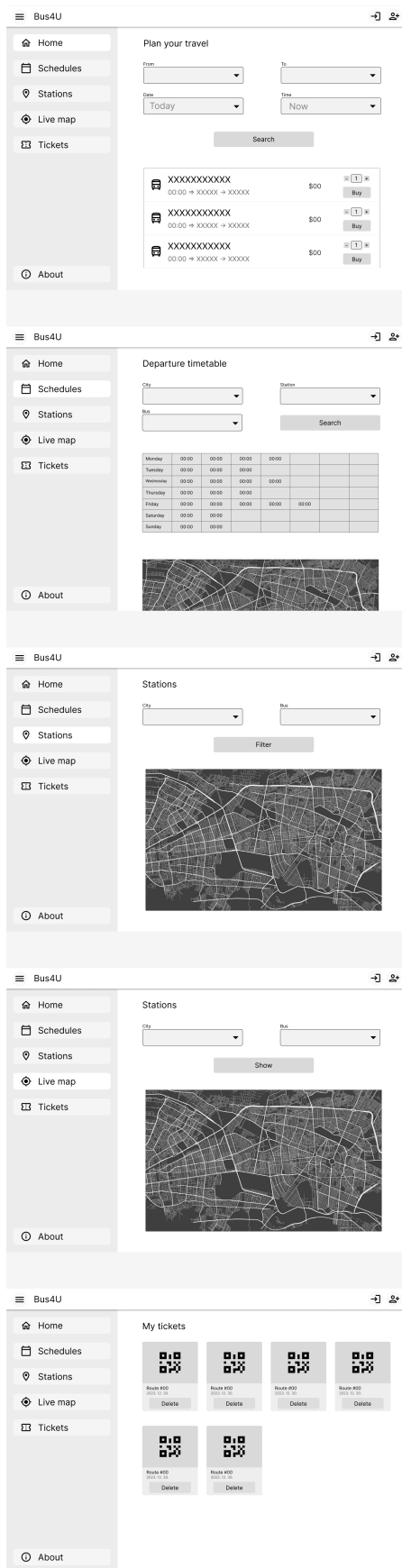
## 4. fejezet

# A felhasználói felület tervezése

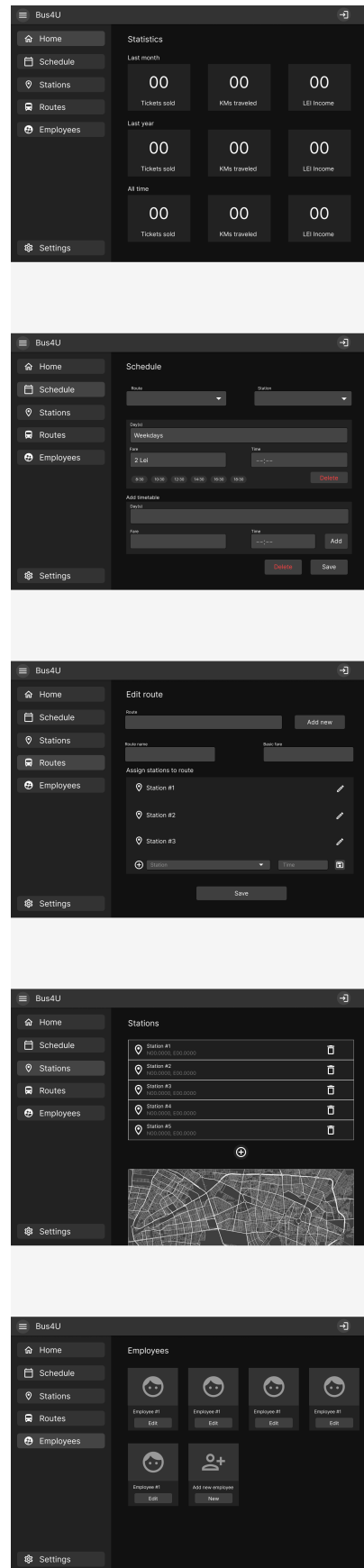
Az egyik legfontosabb része a projektnek a UI, amelynek megtervezésére a Figma segédprogramot használtam, amelyben megépítettem az első elképzelésünket a felületekről. Ezt a wireframe-et követve már egyszerűbb volt a fejlesztés. Munka közben jöttek jobb ötletek is, mint a kezdeti, így a végső termék nem teljesen egyezik meg a tervvel, de iránymutatásnak megfelelt:



4.1. ábra. A mobilalkalmazás terve



4.2. ábra. A felhasználói weboldal terve



4.3. ábra. Az admin weboldal terve

## 5. fejezet

# Gyakorlati megvalósítás

### 5.1. Felhasznált technológiák

A felhasználói és az admin weboldalak Vue.js keretrendszerben készültek, Vuetify komponenseket felhasználva. Ennek a két technológiának a kombinációja modern, esztétikus és gyors weboldalakat eredményezett, amelyeknek a fejlesztése is egyszerű volt.

Az mobil platformokra szánt alkalmazás Flutter technológiával készült, amely által egyetlen kódbázisból ki lehetett generálni Android és iOS készülékekre is a natív alkalmazást.

#### 5.1.1. Vue.js

A Vue.js egy progresszív JavaScript keretrendszer, amelyet azért hoztak létre, hogy a webes felhasználói felületek fejlesztése egyszerűbb és gyorsabb legyen. A JavaScript keretrendszerek célja, hogy megkönnyítsék a webprogramozást, azáltal hogy előre megírt osztály- és függvénykönyvtárakat szolgáltatnak, amelyek segítenek lerövidíteni a fejlesztési időt. Első ilyen keretrendszer a jQuery volt, amely a DOM (Document Object Model) manipulációt és az AJAX (Asynchronous JavaScript And XML) hívásokat tette egyszerűbbé, majd később a React és az Angular keretrendszerek jöttek, amelyek a komponens alapú fejlesztést tették lehetővé. Ez a fejlődés viszont mind nagyobb és nagyobb kiterjedésű csomagokat eredményezett, ami kissé csökkentette a weboldalak teljesítményét. A Vue.js ezt a problémát próbálja megoldani, azzal hogy egy könnyű, gyors és hatékony keretrendszert kínál, amely egy Virtuális DOM modell segítségével gyorsítja a futást és amelynek a forráskódja akár 24Kb-ra is zsugorítható, így viszonylag gyors a betöltése is [2].

A Vue.js két alapköve a deklaratív UI felépítés és a reaktivitás. A deklaratív felépítés azt jelenti hogy az alap HTML kódot a Vue kiegészíti egy saját sablon-szintaxissal, amely lehetővé teszi a változók beágyazását a HTML kódba. A reaktivitás azt jelenti, hogy amikor a HTML sablonban felhasznált JavaScript változók értéke megváltozik, akkor a Vue észleli ezt és automatikusan megváltoztatja a megjelenített HTML kódot, így mindig a legfrissebb adatokat látja a

felhasználó. Ugyanez végbemegy fordított irányban is, mert a felhasználói interakciók eredményeképpen a Vue automatikusan frissíti a háttérben lévő változókat is, így lesz a kommunikáció a DOM és a Vue között kétirányú [2, 1].

A Vue keretrendszer egyik legnagyobb előnye, hogy könnyen tanulható és alkalmazható, így gyorsan lehet vele fejleszteni. Ennek egyik oka, hogy komponens alapú, amely lehetővé teszi a weboldalak felépítését újrafelhasználható, különálló komponensekből, amelyeket később össze lehet fűzni egy nagyobb alkalmazássá. Ezek úgynevezett Single-File Component-ek (egyfájlos-komponensek) formájában jelennek meg, amely azt jelenti, hogy az adott komponens HTML, CSS és JavaScript kódja mind egy fájlban belül található, ezzel egy átlátható és moduláris alkalmazásfelépítést eredményezve [1]. Emellett a Vue nagyon jó online dokumentációval rendelkezik, amelyben minden funkció részletesen le van írva, de különleges problémákkal és kérdésekkel a dinamikusan növekvő fejlesztői közösséghez is lehet fordulni, így mindig könnyen meg lehet találni az éppen szükséges információt és nagyon gyorsan el lehet sajátítani a keretrendszer használatát [2].

A Vue legújabb verziója a Vue 3, amely a komponensek leírására 2 féle API-t (Application Programming Interface) is biztosít: Options és Composition. A Composition API a 3-as verziótól került bevezetésre és különféle teljesítménybeli optimalizációkat tartalmaz a régi Options API-jal szemben, de a szintaxisa is egyszerűbb, mert közelebb áll az alap JavaScript szintaxisához [1]. A Vue 3 a TypeScript támogatását is bevezette, amely egy statikus típusellenőrző nyelv a JavaScript fejlesztéséhez és amely segít a hibák korai felfedezésében, valamint a kód minőségének javításában. Én a munkám során a Vue 3-at és a Composition API-t használtam, mivel ezek a legújabb és legelőnyösebb technológiák jelenleg.

Fontos megemlíteni a Vue ökoszisztémájába tartozó egyéb eszközöket is, amelyek segítik a fejlesztést. Ilyen például a Vue Router, amely a weboldalak közötti navigációt teszi lehetővé, a Pinia, amely a globális állapotkezelést segíti, a Vite, amely a projekt generálását és a fejlesztést segíti, vagy a Vitest amely a teszteléshez szolgáltató hasznos eszközöket [1]. Ezek közül kiemelném a Vite nevű build tool-t (építő eszközt), amelynek a gyors újratöltés funkciója figyeli a forráskódban létrejött módosításokat és azonnal frissíti a weboldalt, így a fejlesztőnek nem kell manuálisan a frissítés gombot nyomogatni minden begépett kódsor után.

A felhasználó szemszögéből a leglátványosabb tulajdonság, hogy nagyon gyors a betöltés és a navigáció, mivel a Vue Single Page Application (Egyoldalas Alkalmazás) üzemmódja és a Vue Router kombinációja által lényegében csak egyszer töltődik be a webalkalmazás, utána az oldalak közötti navigáció során csak a tartalom frissül, nem pedig az egész oldal, így lényegesen kevesebb adat közlekedik a hálózaton keresztül és a böngészőnek is kevesebb dolga van az oldalak megjelenítésénél [1]. Ezeknek a funkcióknak a segítségével egy gyors és hatékony felhasználói felületet lehet készíteni, amely a gyengébb hálózati lefedettséggel és a gyengébb hardverrel rendelkező eszközökön is jól használható.

### 5.1.2. Vuetify

A Vuetify egy Material Design alapú Vue.js-re épülő keretrendszer, amely lehetővé teszi a gyors és egyszerű felhasználói felületek készítését, azáltal hogy rengeteg előre elkészített komponens tartalmaz, amelyeket egyszerűen be lehet illeszteni a weboldalba, így sok időt lehet vele spórolni. A Vuetify segítségével gyorsan és hatékonyan lehet modern és esztétikus felhasználói felületeket készíteni, amelyek jól néznek ki és jól működnek minden eszközön.

### 5.1.3. Flutter

A Flutter egy nyílt forráskódú, Google által fejlesztett cross-platform keretrendszer, amely lényege, hogy egyetlen kódbázisból ki lehet generálni az alkalmazásokat a legnépszerűbb platformokra. Ezek a platformok a következők: Android, iOS, Windows, macOS, Linux, valamint a web. A Flutter a Dart programozási nyelvet használja, amely egy modern, gyors és hatékony nyelv, és amely lehetővé teszi a gyors és hatékony fejlesztést. A Flutter egyik nagy előnye, hogy egy widget rendszert használ, amely lehetővé teszi a felhasználói felületek egyszerű és gyors elkészítését, valamint a különböző platformokra történő kiadást. Ezek a widgetek alapértelmezetten a Material Designra épülnek, így egy bevált és szép felhasználói felület építhető fel belőlük, amely összhangban van a Vuetify webes komponenseivel is. A Flutter egy másik nagy előnye, hogy a natív alkalmazásokhoz hasonlóan gyors és hatékony, mivel natív kódra fordítja le a Dart kódot, így a lehető legjobb teljesítményt nyújtja, bármilyen platformról is legyen szó. Jelen projekt esetében, mivel a webes felületet a Vue.js és a Vuetify keretrendszerekkel készítettem, a Flutter használatakor inkább a mobilos platformokra koncentráltam, mint az Android és az iOS, ezáltal a legnépszerűbb platformok le lettek fedve és bárki, bármilyen eszközről el tudja érni a Bus4U-t.

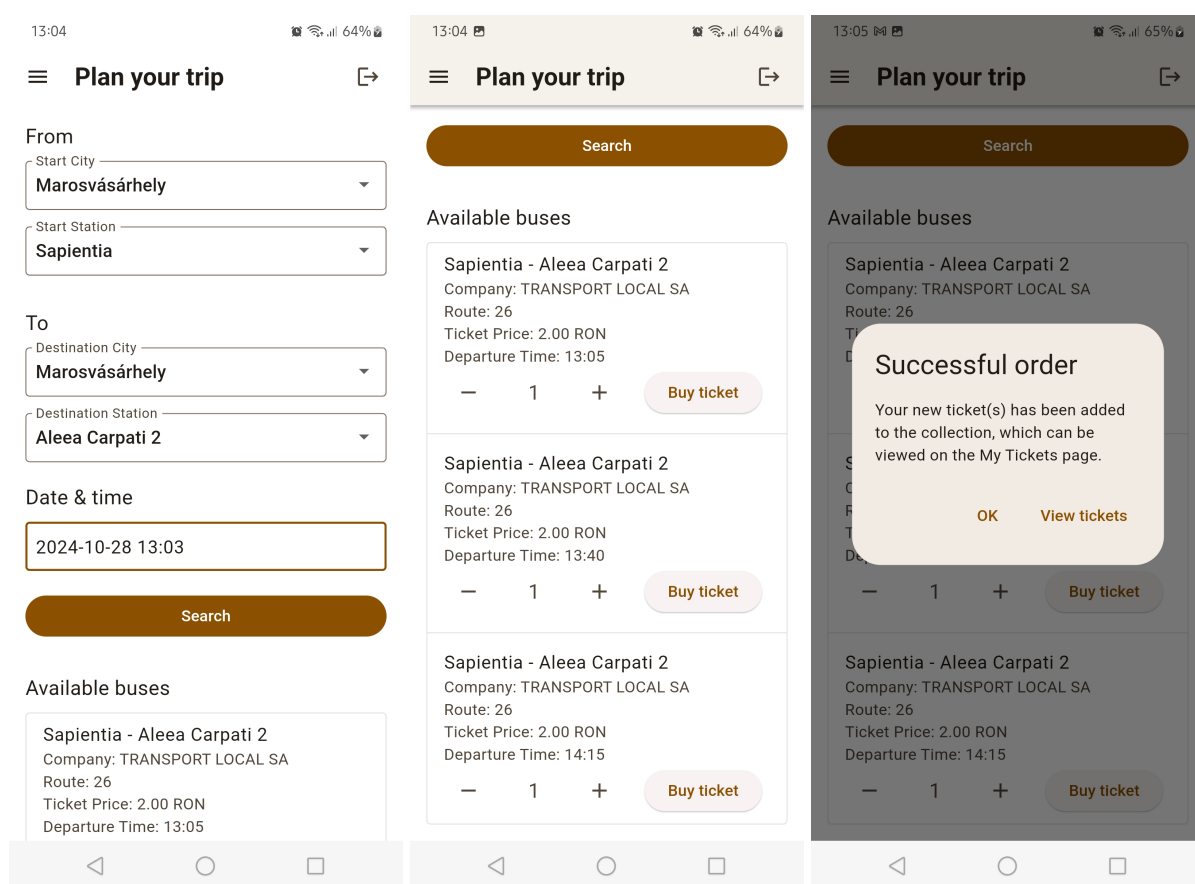
## 5.2. Felhasználói felület

A felhasználói felület működése megegyezik a weboldalon és a mobilalkalmazásban is, azonban a megjelenésük kicsit eltér egymástól, mivel a weboldal a böngészőben fut, míg az alkalmazás a mobil eszközön. A felületek létrehozásakor törekedtem a hasonlóságra, így a navigációs menü, a UI komponensek és a színpaletta is megegyezik, így a felhasználó könnyen átláthatja és használhatja mindkét platformot.

### 5.2.1. Főoldal

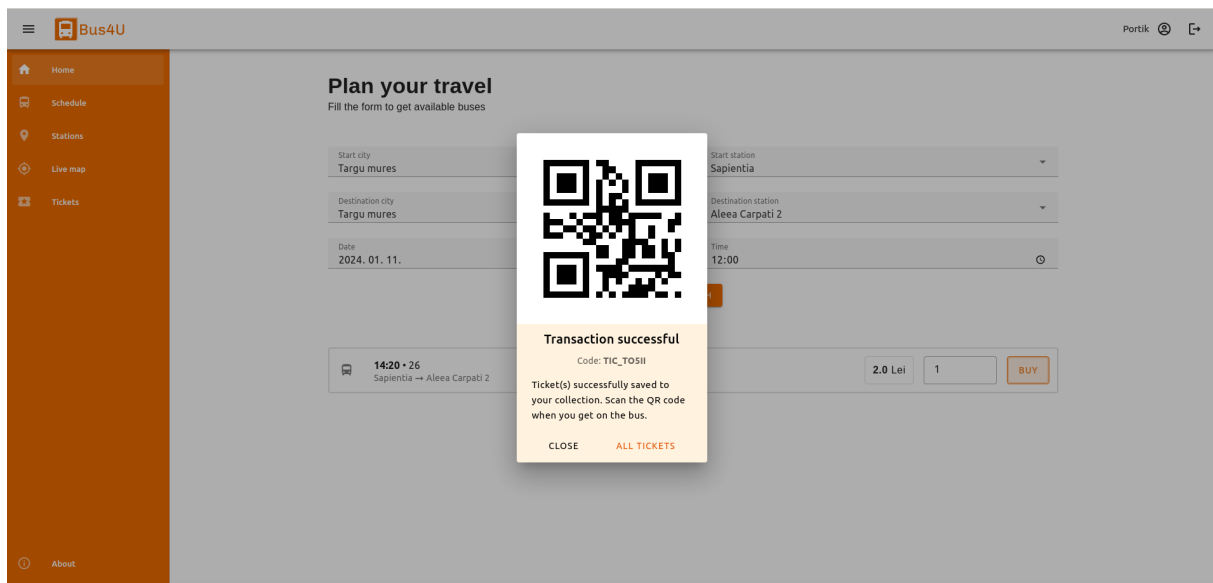
A weboldalra rákeresve a felhasználót a kezdőoldal fogadja, ahol lehetőség van egy utazás megtervezésére. Ehhez ki kell választani a kiindulási helyet és megállót, majd a cél helyét és a megállóját, végül pedig az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a

megadott megállók közötti, az adott napon és a megadott időpont után közlekedő járatokat, valamint azok jegyárát. Minden egyes megjelenő járatnál ki lehet választani a jegyek számát, amely alapértelmezetten egy jegyre van beállítva. Ezt követően a felhasználó, a vásárlás gombra kattintva megszerezheti a jegyet vagy jegyeit az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.



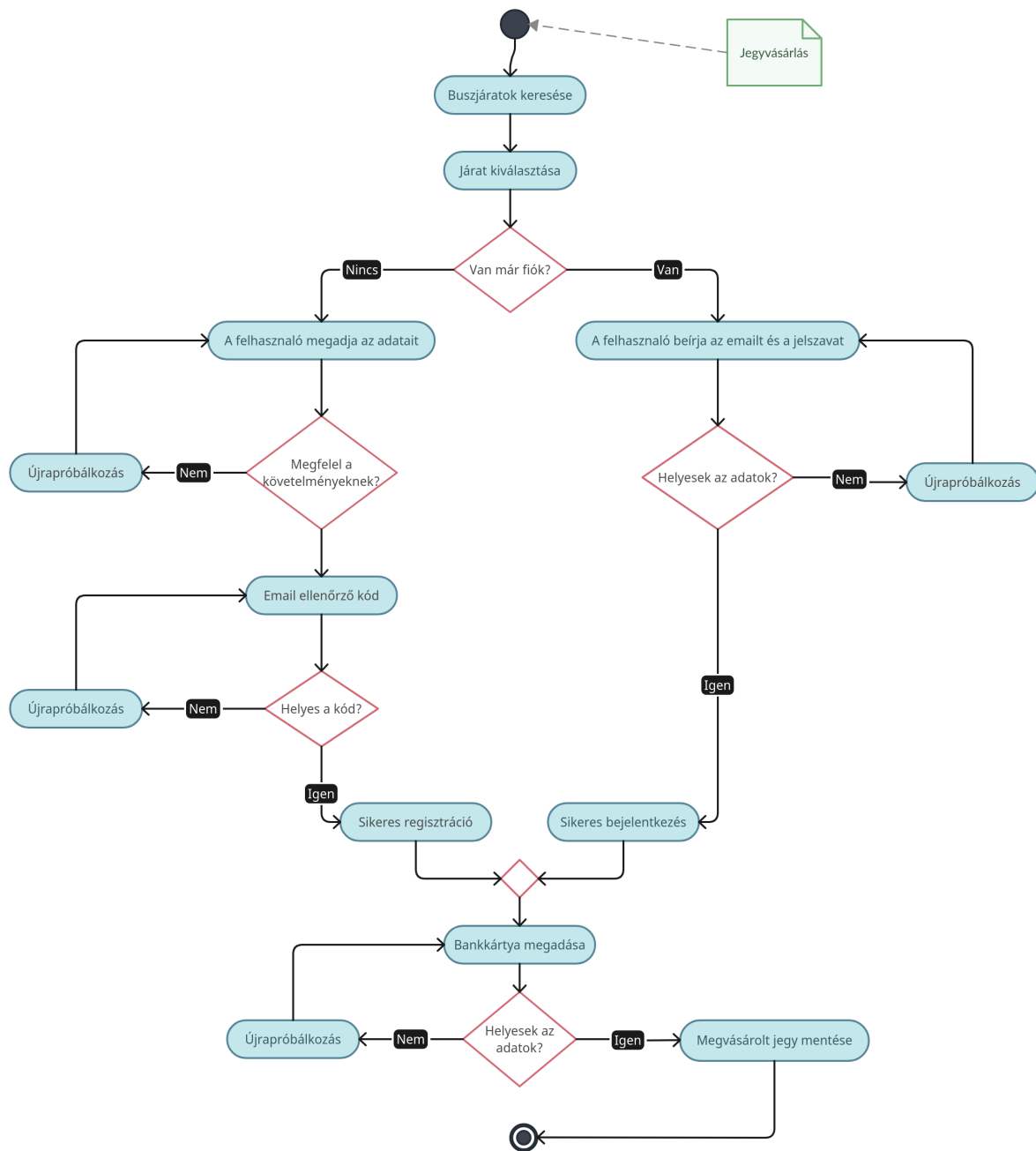
5.1. ábra. Jegyvásárlás a mobilalkalmazásban





5.2. ábra. Jegyvásárlás a weboldalon

A könnyebb megértés végett, a jegyvásárlási és bejelentkezési folyamat szemléltetésére elkészítettem a 5.3 ábrán lévő aktivitás diagramot:



5.3. ábra. A jegyvásárlás folyamata

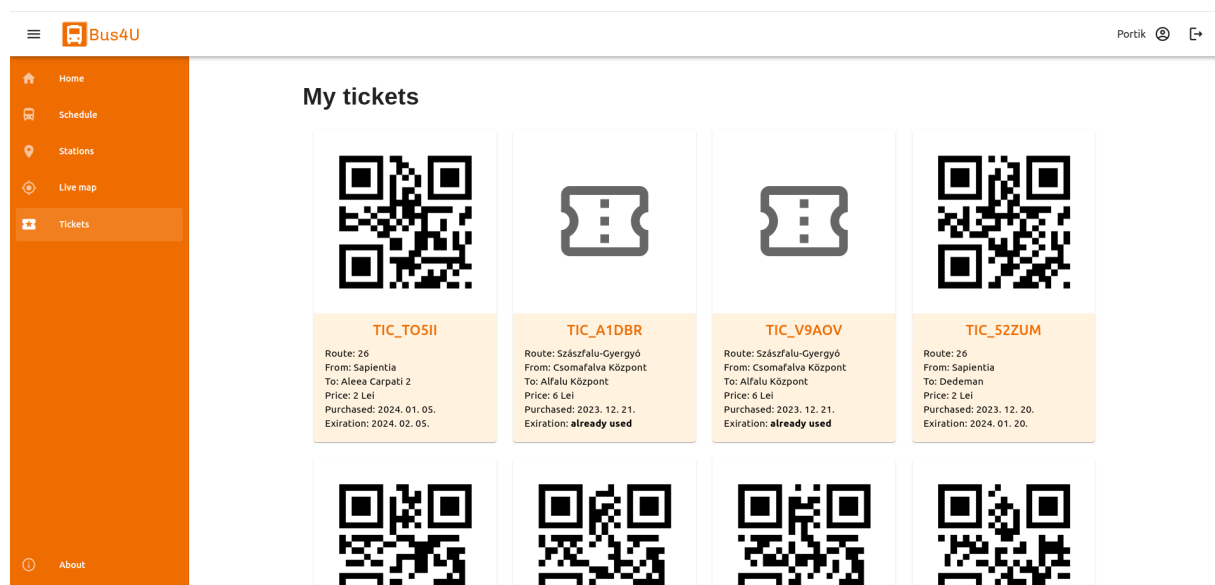
Ahogy az ábrán is látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rákattintott a jegy(ek) megvásárlására, a rendszer átirányítja őt a bejelentkezéshez, itt, ha van már fiókja, bejelentkezhet a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol meg kell adnia a nevét, email címét, és egy új jelszót. A rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó) akkor egy ennek megfelelő hibüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán a szerver

automatikusan küld egy 4 számjegyből álló ellenőrző kódot a megadott e-mail címre, amelyet a felhasználó be kell írjon az alkalmazásban megjelenő szövegmezőbe. Ha ez a kód helytelen, akkor a folyamat kezdődik előlről, a felhasználónak új adatokat kell megadnia. Ha regisztráció sikeres és az ellenőrzőkód helyes, akkor a rendszer átirányítja a felhasználót a bankkártyás fizetési oldalra, ami jelenleg nincs implementálva, ez a jövőbeli tervek között szerepel. Ezután már csak az maradt, hogy a backend generál egy egyedi kódot a megvásárolt jegyhez és megjelenik a kijelzőn a sikeres vásárlás visszaigazolása. Az itt megvásárolt jegye(ke)t a felhasználó a Jegyek nevű oldalon találja meg szöveges és QR-kódos formátumokban.

## 5.2.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a felhasználó gyűjteményébe, amely a Jegyek oldalon található, így ezek bármikor visszakereshetőek és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség. A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve és el van rejtve a rajtuk lévő QR-kód így azokat nem lehet többször felhasználni. A jegyek a létrehozási idejük szerinti csökkenő sorrendben vannak listázva, így a legfrissebb jegyek mindig az elején vannak.

A itt megjelenő jegyeket úgy lehet felhasználni, hogy a buszra való felszálláskor a sofőr mellett elhelyezett POS terminálra kell mutatni az aktuális jegy QR-kódját, az pedig leolvassa és elküldi a szervernek ellenőrzésre. Ha a kód érvényes és minden rendben van, akkor a felhasználó felszállhat a buszra, a jegy pedig invalidálódik.



5.4. ábra. A felhasználó megvásárolt jegyei

### 5.2.3. Menetrend

A következő oldal a Menetrend, amely hasonlóan az egyes buszmegállókban kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból, napokra lebontva. A mi megoldásunk viszont tartalmaz emellett egy interaktív térképet is, ahol a felhasználó meg tudja tekinteni a kiválasztott járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő helyzeteit is, a saját pozíciója mellett.

Bus4U

Portik

## Schedule

Select a station and a bus to view the departure timetable and the route marked on the map

City  
Marosvásárhely

Station  
Dedeman

Bus  
Gyergyó-Marosvásárhely

SEARCH

|                                                                      |       |       |
|----------------------------------------------------------------------|-------|-------|
| Departure times for station: Dedeman and bus: Gyergyó-Marosvásárhely |       |       |
| Monday                                                               | 07:45 | 19:45 |
| Tuesday                                                              | 07:45 | 19:45 |
| Wednesday                                                            | 07:45 | 19:45 |
| Thursday                                                             | 07:45 | 19:45 |
| Friday                                                               | 07:45 | 19:45 |

☒ Current location☒ Live bus location☒ Stations

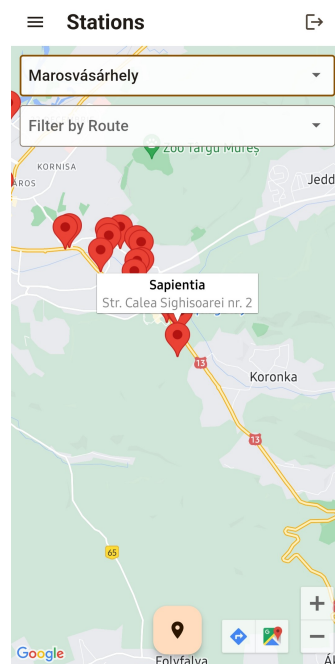
5.5. ábra. Listázott menetrend és az útvonal térképe

## 5.2.4. Megállók és élő térkép

Ez a két oldal az előbb említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. A Megállók oldal megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat szerint. Az egyes megállókra kattintva a térképen megjelenik egy kis buborék a kiválasztott megálló adataival, mint a név és a cím, így könnyebb megtalálni a keresett megállót.

Az élő térkép oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ezeket az adatokat a buszokon lévő POS terminálok szolgáltatják, így biztosak lehetünk abban, hogy a buszok helyzetei valós időben jelennek meg. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a megállóban, ha egy busz késik, így időt lehet vele spórolni.

Mindkét oldalon megjelenik a felhasználó jelenlegi pozíciója, amennyiben engedélyezte az ehhez való hozzáférést a telefonján, így könnyen megtalálhatja a legközelebbi megállót vagy buszt.



5.6. ábra. Megállók

## 5.3. Adminisztrátori weboldal

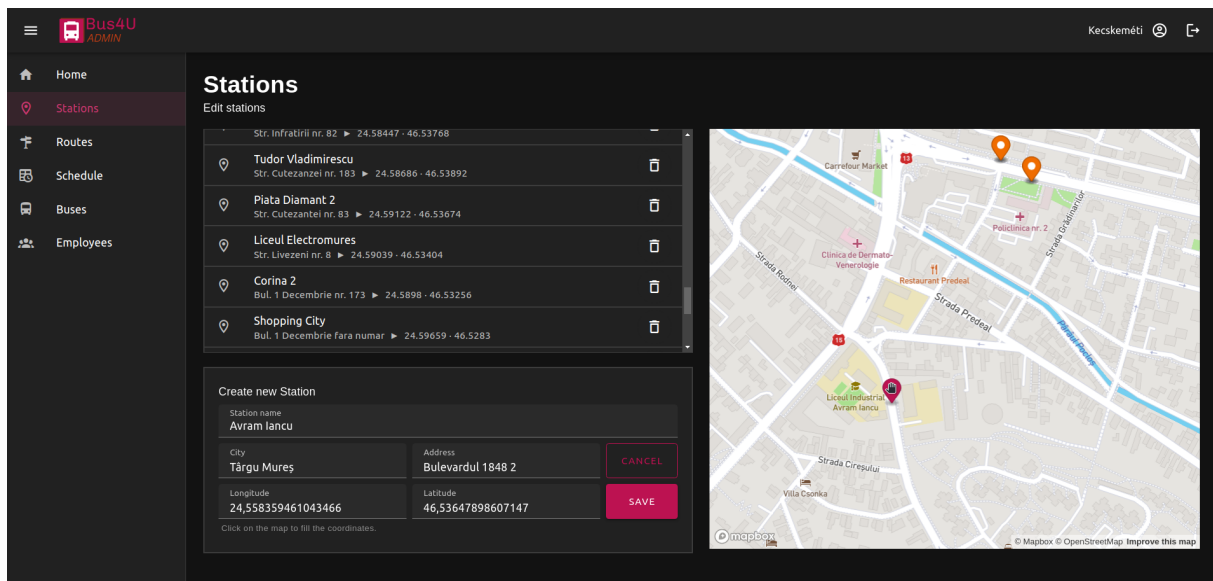
Az admin felület csak bejelentkezéssel elérhető, ehhez egy admin rangú felhasználói fiók szükséges, amely egy adott céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden admin csak a saját cégéhez tartozó adatokat láthatja és módosíthatja.

### 5.3.1. Kezdőoldal/Statisztikák

Belépés után egy összegző oldal jelenik meg amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindez havi, évi és mindenkor bontásban. Itt jelennek meg az esetleges figyelmeztetések és értesítések is, például a buszok lejárt útheadója vagy a közelgő műszaki vizsgálat.

### 5.3.2. Megállók

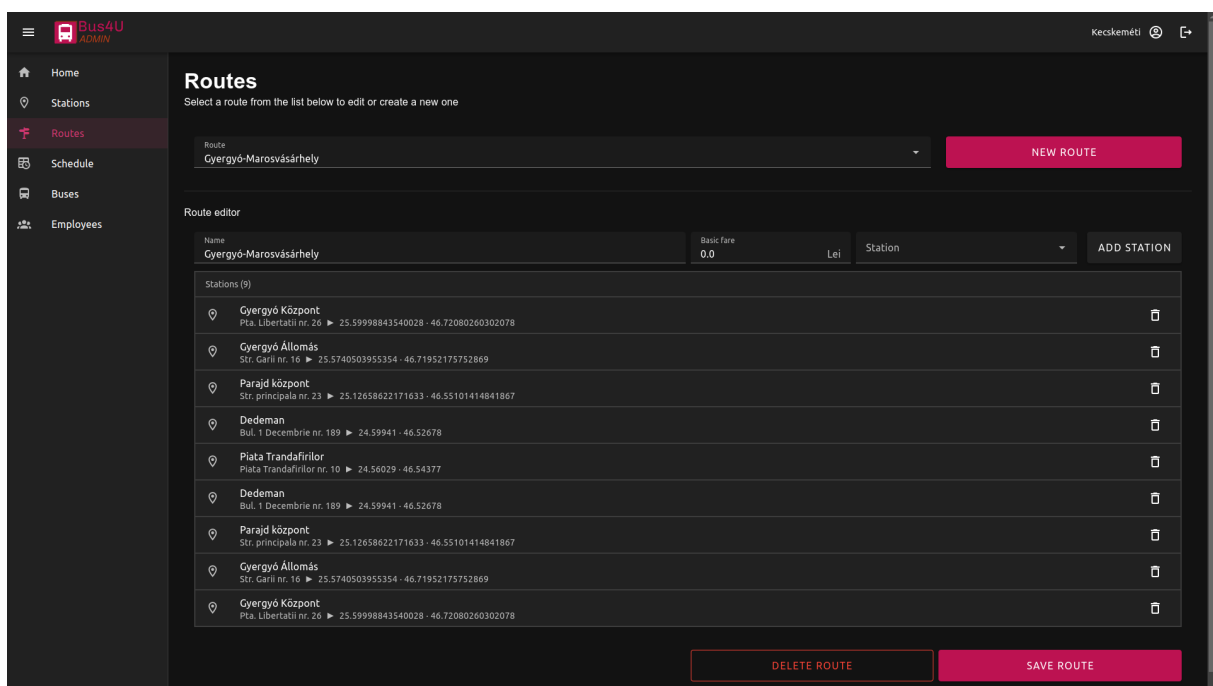
A Megállók oldalon létre lehet hozni új megállókat, megtekinteni meglévőket a térképen vagy törölni ezeket. Új megálló hozzáadásához meg kell adni a megálló nevét, a címet és a koordinátákat. A néven kívül, a többi adatot automatikusan kitölti a rendszer, ha a térképen a megálló helyére kattint a felhasználó, de manuálisan is meg lehet adni.



5.7. ábra. Megállók létrehozása, megtekintése és törlése

### 5.3.3. Útvonalak

Ezen az oldalon lehet felépíteni útvonalakat a létrehozott megállókból és itt lehet megadni az alap jegyárat is, abban az esetben, ha a jegyek ára nem függ a megtett távolságtól és időtől, hanem mindig és mindenhol ugyanannyi, ez esetben a fizetés az útvonalhoz lesz hozzárendelve, nem a különböző megállókhoz.



5.8. ábra. Útvonalak létrehozása, módosítása és törlése

### 5.3.4. Menetrend

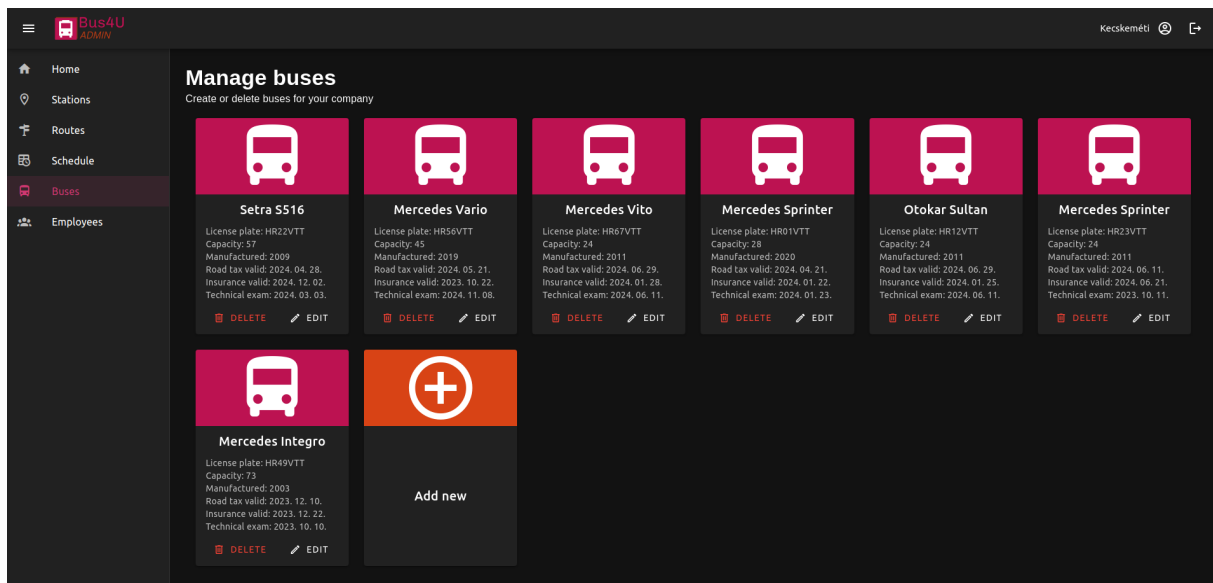
A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt meg kell adni az órarendre érvényes napokat, a következő megállóig szükséges jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, ha különböző napokra különböző órarend érvényes. Így el lehet választani a hétvégét a hétköznapoktól, ha mások az indulási időpontok, vagy ha például hétvégén drágább a jegy.

The screenshot shows the 'Schedule' management interface in the Bus4U ADMIN system. The left sidebar contains navigation links: Home, Stations, Routes, Schedule (active), Buses, and Employees. The main content area is titled 'Schedule' with the instruction 'Select a route and a start station to add/edit timetables'. It features three dropdown menus for 'Route' (Városfalva-Udvarhely), 'From station' (Homoródszentpéter), and 'To station' (Homoródszentmárton). Below these, there are two sections for adding or editing timetables. The first section is for 'Monday, Tuesday, Wednesday, Thursday, Friday' with a fare of 3 and times 06:30, 11:45, 14:30, and 17:00. The second section is for 'Saturday, Sunday' with a fare of 4 and times 10:00 and 14:00. Each section has buttons for 'DELETE', 'CLEAR', 'ADD TIME', and 'SAVE DAY(S)'. At the bottom, there are 'CANCEL EDITING' and 'SAVE SCHEDULE' buttons.

5.9. ábra. Menetrend létrehozása/törlése

### 5.3.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Hasonlóan működnek, mindkettőn található egy lista a meglévő elemekkel, ahol bármelyiket lehet módosítani vagy törölni, illetve újat is lehet létrehozni. A buszoknál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdó, a biztosítás és a műszaki engedély lejárat dátumát. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást, telefonszámot, lakcímet (opcionális), és egy új jelszavat.



5.10. ábra. Buszok kezelése



## 6. fejezet

# Összefoglalás

Befejezésképpen bátran kijelenthetem, hogy a projekt elején leszögezett terveket sikerült implementálni, és fejlesztés közben egyéb hasznos funkcionalitásokat is sikerült hozzáadni a végső eredményhez. Ezáltal a végfelhasználóknak egy-egy funkciódús és jól működő weboldal és applikáció áll a rendelkezésükre, amely nagyban megkönnyíti a tömegközlekedésben való részvételt, bármilyen eszközön is próbálnák elérni ezt. A busztársaságoknak pedig egy egyszerű vezérlőfelület van biztosítva, amely segítségével könnyen és gyorsan tudják kezelni a buszokat, az útvonalakat, a menetrendeket, a jegyárakat és az alkalmazottakat, mindet egy helyen.

Véleményem szerint a Bus4U további idő és munka befektetésével, egy reális piaci rést tud betölteni, mivel a buszokkal való közlekedés elengedhetetlen a forgalom, és a környezet-szennyezés csökkentéséhez, de jelenleg, a rendszer hiányosságai miatt ez egy nem túl népszerű közlekedési eszköz.

### 6.1. További fejlesztési lehetőségek

Felhasználók szempontjából a következő fontos fejlesztés a buszok telítettségének valós idejű követése lenne, mert ezzel elkerülhető lenne a zsúfoltság a buszokon, és így sokat javulna a tömegközlekedés minősége. Be szeretnénk idővel vezetni a bérletek, illetve különböző kedvezmények kezelésének lehetőségét is, mert ez is növelheti a népszerűséget.

A busztársaságok szempontjából még több lehetőség áll rendelkezésre. Mivel a projektet úgy terveztük, hogy több busztársaság is fel tudja használni, így ez jelen pillanatban általános használatra van optimalizálva. Ennek ellenére egy-egy partner cég kérésére személyre is szabhatnánk a rendszert, egyedi funkciókat és kinézetet bevezetve. Az is hasznos lenne például, ha a cég akár a teljes könyvelését le tudná itt bonyolítani: az alkalmazottak munkáóráit automatikusan vezetni, az autóbuszok tankolásait összesíteni, illetve buszokon lévő POS terminál GPS moduljának segítségével akár a Foaie de parcsurs vezetése is automatizálható lenne. A könyveléshez a járművek szervízköltségének számláit is kellene gyűjteni az oldalra, amivel további hasznos kimutatásokat tudnánk generálni.

# Irodalomjegyzék

- [1] John Au-Yeung. *Vue.js 3 By Example: Blueprints to learn Vue web development, full-stack development, and cross-platform development quickly*. Packt Publishing Ltd, 2021.
- [2] Erik Hanchett and Ben Listwon. *Vue.js in Action*. Simon and Schuster, 2018.